

SINGLE SIGN-ON (SSO)

Do zero à arquitetura, implementação, segurança e operação

Guia aprofundado para desenvolvedor

OAuth 2.0 • OpenID Connect • SAML 2.0 • Active Directory / Entra ID • AWS STS / IAM Identity Center •
SCIM • JWT • sessões • offline • segurança • testes

Atualizado em 18 de agosto de 2026

Objetivo Ao terminar este material, você deve conseguir explicar o que é SSO, diferenciar autenticação de autorização, desenhar os principais ciclos OIDC e SAML, entender onde AD/Entra e AWS entram, discutir funcionamento durante indisponibilidade e participar de decisões de implementação sem depender apenas de palavras-chave.

Como usar este guia

Você comentou que nunca tinha ouvido falar de SSO. Por isso, o material começa por conceitos básicos e cresce em camadas. Não tente decorar siglas na primeira leitura. A prioridade é entender quem confia em quem, qual artefato prova a identidade, qual artefato concede acesso e onde cada sessão existe.

Roteiro sugerido para uma tarde de estudo

Bloco	Tempo sugerido	Foco	Resultado esperado
1. Fundamentos	60-75 min	Seções 1 a 5	Explicar SSO, IdP, SP/RP, federação, autenticação vs autorização e token.
2. Protocolos	90-120 min	Seções 6 a 10	Desenhar Authorization Code + PKCE, OIDC, SAML e o ciclo de sessão/logout.
3. Ecossistema corporativo	60-90 min	Seções 11 a 15	Entender AD/Entra, AWS, STS, IAM Identity Center, Cognito e SCIM.
4. Engenharia	90-120 min	Seções 16 a 24	Discutir offline, arquitetura, segurança, testes, observabilidade e plano de implementação.

Regra de ouro SSO é um comportamento/resultado de arquitetura, não um protocolo. O SSO pode ser realizado com OIDC, SAML, Kerberos e outras tecnologias. OAuth 2.0, sozinho, é um framework de autorização - não um protocolo de login. [R1][R4]

O que seu tech leader provavelmente quis dizer

Sua anotação	Provável termo	Confiança	Por que é relevante
“alphi 2”	OAuth 2.0	Alta	É a base moderna para autorização e serve de base para OpenID Connect.
“AD”	Active Directory (AD DS)	Alta	Diretório corporativo on-premises, fortemente ligado a Kerberos, LDAP, usuários, grupos e computadores.
“exagure”	Azure / Azure AD, hoje Microsoft Entra ID	Média-alta	Azure AD foi renomeado para Microsoft Entra ID; é um IdP/serviço de identidade cloud muito comum em SSO empresarial. [R11]
“AWS teste”	AWS STS ou AWS SSO/IAM Identity Center	Média	STS emite credenciais temporárias; AWS SSO foi o nome anterior do serviço hoje chamado IAM Identity Center. Ambos aparecem em conversas sobre federação/SSO AWS. [R12] [R14]

Como não temos a gravação da conversa, trate essas correspondências como hipóteses. O guia cobre todas elas de forma suficiente para você reconhecer o assunto quando conversar novamente com seu tech leader.

1. O modelo mental de identidade

1.1 Identidade, conta, credencial e principal

Identidade é a representação de uma pessoa, serviço, dispositivo ou workload dentro de um domínio de confiança. Uma conta é um registro dessa identidade em algum diretório ou aplicação. Uma credencial é algo usado para provar a identidade - senha, chave privada, certificado, passkey, código de MFA, ticket Kerberos. Um principal é a entidade que efetivamente recebe permissões e executa ações.

Conceito	Pergunta que responde	Exemplo
Identidade	Quem é?	Maria da empresa ACME; serviço billing-api; runner de CI.
Conta	Onde essa identidade está registrada?	maria@acme.com no Entra ID.
Credencial	Como prova que é ela?	Passkey + MFA; certificado; senha.
Principal	Quem está fazendo esta requisição?	sub=9f... em um token; IAM role assumida.
Atributo/claim	O que sabemos sobre ela?	department=finance, groups=[admins], tenant=acme.
Permissão	O que pode fazer?	invoice.read; atualizar usuário; assumir role X.

1.2 Autenticação não é autorização

Autenticação (authentication, frequentemente abreviada como AuthN) responde “quem é você?”. Autorização (authorization, AuthZ) responde “o que você pode fazer?”. Um usuário pode estar perfeitamente autenticado e ainda receber HTTP 403 porque não tem permissão para a operação.

Erro clássico Usar o access token como “prova de login” sem entender sua finalidade, ou usar o ID token para chamar uma API. Em OIDC, o ID token comunica ao cliente informações sobre a autenticação; o access token é destinado ao recurso/API. [R4]

1.3 Fonte de identidade, IdP e diretório

Um diretório armazena identidades e atributos. Um Identity Provider (IdP) autentica usuários e emite uma afirmação/token que outras aplicações confiam. Em alguns produtos as duas funções aparecem no mesmo ecossistema, mas conceitualmente são diferentes. Active Directory Domain Services é um diretório corporativo on-premises; Microsoft Entra ID é o serviço de identidade e acesso multicloud da Microsoft e não é simplesmente “o AD na nuvem”. [R11]

2. O que Single Sign-On realmente significa

Single Sign-On significa que o usuário estabelece uma sessão de autenticação com uma autoridade confiável e, a partir disso, consegue acessar múltiplas aplicações sem fornecer novamente a credencial primária a cada aplicação. A experiência pode parecer “entrei uma vez e tudo abriu”, mas por baixo há várias sessões e vários tokens independentes.

2.1 O que SSO resolve

- **Menos senhas espalhadas:** as aplicações deixam de armazenar ou verificar a senha corporativa.
- **Política centralizada:** MFA, bloqueio, risco, horário, dispositivo e regras podem ser concentrados no IdP.
- **Offboarding mais rápido:** desabilitar uma identidade central pode cortar novas autenticações em muitos sistemas.
- **Auditoria:** há um ponto central para eventos de autenticação, embora cada aplicação continue precisando auditar autorização e ações.
- **Experiência do usuário:** menos prompts repetidos e menos necessidade de memorizar credenciais.

2.2 O que SSO NÃO resolve sozinho

- Não define automaticamente as permissões dentro da sua aplicação.
- Não cria nem remove usuários sozinho. Provisionamento costuma usar SCIM ou mecanismos próprios.
- Não garante Single Logout perfeito em todos os protocolos/aplicações.
- Não elimina sessões locais. A sua aplicação normalmente ainda mantém estado de sessão ou tokens próprios.
- Não significa que a aplicação deva confiar cegamente em qualquer token “parecido com JWT”.

2.3 Sessão central versus sessão da aplicação

```

Usuário
|
| 1) autentica uma vez
v
[ IdP / Authorization Server ] ---- sessão central do IdP
|
| 2) emite assertion/token para App A
+-----> [ App A ] ---- sessão local A
|
| 3) depois emite assertion/token para App B, sem nova senha
+-----> [ App B ] ---- sessão local B

```

SSO acontece porque, ao chegar ao IdP pela segunda vez, o navegador já possui uma sessão válida com ele. O IdP pode emitir nova resposta para a segunda aplicação sem pedir senha novamente, sujeito a política, MFA, risco, prompt e tempo desde a última autenticação.

3. Federação e confiança

Federação de identidade é o estabelecimento de confiança entre domínios diferentes. Em vez de a aplicação validar a senha do usuário, ela confia em uma afirmação assinada emitida por um IdP. Essa confiança precisa ser configurada: quem é o emissor aceito, para qual aplicação o token é destinado, quais chaves/certificados são válidos, quais URLs de retorno são permitidas e quais claims devem existir.

3.1 Os atores que você verá

Termo	OIDC/OAuth	SAML	Papel
Usuário	Resource Owner / End-User	Principal/Subject	Pessoa que está se autenticando.
Aplicação	Client / Relying Party (RP)	Service Provider (SP)	Sistema que quer identificar o usuário.
IdP	Authorization Server / OpenID Provider (OP)	Identity Provider (IdP)	Autentica e emite token/assertion.
API	Resource Server	Nem sempre aparece no SAML web SSO	Recurso que aceita access token e aplica autorização.

3.2 Trust não é “a assinatura bateu”

Validar assinatura é apenas uma parte. O consumidor deve validar issuer, audience, validade temporal, nonce/state quando aplicável, algoritmo permitido, chave correta, redirect URI, contexto do fluxo e claims necessárias. Um token assinado por uma chave legítima ainda pode ser inválido para a sua aplicação se tiver outra audience ou tiver sido emitido para outro contexto. [R2][R9]

4. Tokens, assertions, cookies e sessões

4.1 Quatro coisas que não devem ser confundidas

Artefato	Quem costuma emitir	Para que serve	Duração típica
Cookie de sessão do IdP	IdP	Manter sessão central de autenticação no navegador.	Política do IdP.
Cookie de sessão da aplicação	Aplicação/BFF	Manter sessão local da aplicação.	Minutos/horas, renovável.
ID Token (OIDC)	OpenID Provider	Informar ao cliente sobre autenticação e identidade.	Curta.
Access Token	Authorization Server	Autorizar acesso a API/recurso.	Curta.
Refresh Token	Authorization Server	Obter novos tokens sem presença do usuário.	Mais longa, com políticas/rotação.
SAML Assertion	SAML IdP	Afirmações sobre autenticação/atributos para um SP.	Muito curta; consumida no login.

4.2 JWT é um formato, não um mecanismo de segurança completo

JWT (JSON Web Token) é um formato compacto de claims. Pode aparecer como ID token e também como access token, mas um access token não é obrigado a ser JWT. O fato de uma string ter três blocos separados por ponto não significa que seja segura ou válida. O consumidor precisa fazer validação criptográfica e semântica conforme o protocolo. [R8][R9]

```
header.payload.signature
```

Exemplo de payload DIDÁTICO:

```
{
  "iss": "https://idp.example.com",
  "sub": "248289761001",
  "aud": "my-web-app",
  "exp": 1787072400,
  "iat": 1787068800,
  "nonce": "n-0S6_wzA2Mj"
}
```

Claims importantes

Claim	Ideia	Erro comum
iss	Quem emitiu o token.	Aceitar token de issuer inesperado.
sub	Identificador do sujeito no emissor.	Usar e-mail como chave imutável de usuário.
aud	Para quem o token foi emitido.	Não validar audience.
exp / nbf / iat	Janela temporal.	Aceitar token expirado “porque o IdP está offline”.
nonce	Liga resposta OIDC à requisição de autenticação.	Não validar em fluxos onde é necessário.
scope / roles / groups	Capacidades/atributos concedidos.	Confundir claim externa com permissão interna sem política.

4.3 JWS, JWK e JWKS

JWS é a estrutura de assinatura; JWK representa uma chave em JSON; JWKS é um conjunto de chaves. Em OIDC, o cliente normalmente descobre um `jwtks_uri` e usa as chaves públicas para verificar assinaturas. Rotação de chaves significa que você precisa suportar mais de uma key ID (kid), cache controlado e atualização segura. [R4][R9]

5. OAuth 2.0: autorização delegada

OAuth 2.0 é um framework de autorização. Ele permite que um cliente obtenha acesso limitado a um recurso HTTP sem receber a senha do usuário. O resultado central é um access token. OpenID Connect adiciona uma camada padronizada de identidade/autenticação sobre OAuth 2.0. [R1][R4]

5.1 Os papéis do OAuth

Papel	Responsabilidade
Resource Owner	Entidade capaz de conceder acesso ao recurso; frequentemente o usuário.
Client	Aplicação que solicita acesso.
Authorization Server	Autentica/autoriza e emite tokens.
Resource Server	API que aceita access token e protege recursos.

5.2 Authorization Code Flow

Para aplicações web modernas, o fluxo de código de autorização é a base. O navegador recebe apenas um código temporário; o código é então trocado por tokens no token endpoint. PKCE adiciona uma prova para impedir que um código interceptado seja reutilizado por outro cliente. As práticas atuais recomendam Authorization Code + PKCE em muitos tipos de aplicações modernas. [R2][R3][R11]

```

1. Browser -> App: GET /login
2. App -> Browser: 302 https://idp/authorize?
   response_type=code
   client_id=app123
   redirect_uri=https://app/callback
   scope=openid profile email
   state=RANDOM
   nonce=RANDOM
   code_challenge=HASH(verifier)
   code_challenge_method=S256

3. Browser -> IdP: usuário autentica / IdP aplica MFA/políticas
4. IdP -> Browser -> App callback: ?code=ABC&state=RANDOM
5. App valida state
6. App -> IdP /token: code=ABC + code_verifier=ORIGINAL
7. IdP -> App: id_token + access_token (+ refresh_token, conforme caso)
8. App valida ID token, cria sessão local
9. App/API usa access token somente para a audience correta

```

5.3 state, nonce e PKCE: cada um resolve um problema diferente

Mecanismo	Protege principalmente contra	Onde aparece
state	CSRF/mistura de respostas e correlação da transação.	Request e callback de autorização.
nonce	Replay/mistura do ID token em OIDC.	Request OIDC e claim do ID token.
PKCE	Roubo/interceptação do authorization code.	code_challenge no /authorize e verifier no /token.

5.4 Flows que você precisa reconhecer

Flow	Uso típico	Comentário
Authorization Code + PKCE	Login/autorização interativa de web, SPA, mobile, desktop.	Principal fluxo a dominar.
Client Credentials	Serviço para serviço sem usuário.	Não é SSO de usuário. A identidade é a aplicação/workload.
Device Authorization	Dispositivo com entrada limitada, CLI, TV.	Usuário autoriza em outro dispositivo.
Refresh Token	Renovação de acesso sem refazer todo login.	Requer política de armazenamento, rotação/revogação.
Implicit	Legado.	As práticas atuais preferem code + PKCE; evite novo desenho baseado em implicit. [R2]
Resource Owner Password Credentials	Legado/anti-pattern.	Exige capturar senha na app; a BCP atual desaconselha esse modelo. [R2]

5.5 Access token: opaco ou JWT

A aplicação cliente não deve presumir que todo access token é JWT nem acoplar lógica ao conteúdo de um token que pertence a outra API. O resource server conhece o contrato do token que recebe. Dependendo do provedor, tokens podem ser opacos e validados por introspecção ou estruturados e validados localmente.

6. OpenID Connect (OIDC): login moderno sobre OAuth 2.0

OIDC adiciona identidade ao fluxo OAuth. O cliente solicita o scope openid, recebe um ID Token e pode, quando suportado, consultar o UserInfo endpoint. OIDC define discovery, claims, ID token, autenticação, UserInfo e extensões de sessão/logout. [R4]

6.1 Discovery

Um OpenID Provider normalmente publica metadados em um documento de discovery. Ali aparecem issuer, authorization_endpoint, token_endpoint, jwks_uri, userinfo_endpoint, end_session_endpoint e capacidades suportadas. Isso reduz configuração manual e ajuda a suportar rotação e interoperabilidade.

```
GET https://idp.example.com/.well-known/openid-configuration

{
  "issuer": "https://idp.example.com",
  "authorization_endpoint": "https://idp.example.com/oauth2/authorize",
  "token_endpoint": "https://idp.example.com/oauth2/token",
  "jwks_uri": "https://idp.example.com/.well-known/jwks.json",
  "userinfo_endpoint": "https://idp.example.com/oauth2/userinfo",
  "end_session_endpoint": "https://idp.example.com/logout"
}
```

6.2 ID Token versus UserInfo

O ID token contém claims ligadas à autenticação e ao sujeito. O UserInfo é um endpoint protegido por access token que pode devolver claims adicionais do usuário. Nem toda aplicação precisa chamar UserInfo; isso depende do conjunto de claims e do provedor.

6.3 Scopes OIDC comuns

Scope	Propósito
openid	Ativa OIDC. Sem ele, a requisição é OAuth, não uma requisição OIDC.
profile	Solicita claims de perfil padronizadas.
email	Solicita e-mail e, quando suportado, estado de verificação.
offline_access	Solicita refresh token para uso quando o usuário não está presente. Não significa que o IdP deixou de ser necessário para sempre. [R4]

“offline_access” não é “login offline” O scope offline_access pede um refresh token que permite obter novos access tokens sem o usuário presente. A troca do refresh token normalmente ainda exige comunicação com o token endpoint do Authorization Server. [R4]

7. SAML 2.0: SSO empresarial baseado em XML

SAML 2.0 é uma família de padrões para troca de informações de autenticação e atributos entre parceiros de segurança. Em web SSO empresarial, o IdP emite uma SAML Response contendo uma Assertion; o Service Provider valida a resposta e cria sua sessão. [R7]

7.1 SP-initiated SSO

```

Usuário -> SP (sua aplicação)
  1) tenta acessar recurso protegido
SP -> Browser -> IdP
  2) AuthnRequest (geralmente Redirect)
IdP
  3) autentica usuário / MFA / política
IdP -> Browser -> SP ACS
  4) HTTP POST com SAMLResponse assinada
SP
  5) valida XML signature, issuer, audience, destination,
    InResponseTo, timestamps, certificado etc.
  6) mapeia NameID/attributes e cria sessão local

```

7.2 IdP-initiated SSO

No IdP-initiated SSO, o usuário parte do portal do IdP e clica na aplicação. O SP recebe uma resposta sem ter iniciado a AuthnRequest. Isso pode ser conveniente, mas reduz algumas correlações do fluxo; decisões de suporte devem seguir as capacidades e requisitos de segurança do produto/provedor.

7.3 Vocabulário SAML

Termo	Significado
Entity ID	Identificador lógico do IdP ou SP.
ACS (Assertion Consumer Service)	Endpoint do SP que recebe a SAML Response.
SAML Response	Envelope de resposta do IdP.
Assertion	Conjunto de statements/atributos/condições sobre o sujeito.
NameID	Identificador do sujeito conforme formato negociado.
Metadata XML	Documento com entity ID, endpoints, certificados e capacidades.
RelayState	Estado/retorno associado à navegação.
Single Logout (SLO)	Mecanismos SAML para logout coordenado, com limitações operacionais.

7.4 OIDC versus SAML

Critério	OIDC	SAML 2.0
Formato	JSON/JWT + HTTP	XML + bindings HTTP
Aplicações modernas	Excelente encaixe	Possível, mas mais pesado
Legado enterprise/SaaS	Muito comum hoje	Extremamente comum
APIs	Integra naturalmente com OAuth access tokens	Não foi desenhado como protocolo de autorização de APIs
Mobile/SPA	Adequado com flows modernos	Geralmente menos natural
Configuração	Discovery e client registration	Metadata, entity IDs, ACS, certificados

Na documentação atual do Microsoft Entra ID, SAML e OIDC são apresentados como as duas opções principais de protocolo para SSO de aplicações, com OIDC normalmente mais simples para frameworks modernos e SAML ainda muito presente em integrações enterprise. [R11]

8. SSO não é provisionamento: JIT, SCIM e lifecycle

Autenticar alguém não resolve como o usuário aparece no banco da aplicação nem como seus grupos são atualizados. Por isso, identidade corporativa costuma separar login/federação de provisioning.

8.1 Just-in-Time (JIT) provisioning

No primeiro login, a aplicação cria ou atualiza o usuário local com base nas claims recebidas. É simples, mas só acontece quando o usuário faz login. Offboarding e alterações de grupo podem ficar atrasados se dependerem apenas de JIT.

8.2 SCIM 2.0

SCIM é um protocolo HTTP padronizado para gerenciamento de identidades entre domínios. Em cenários corporativos, o IdP/provisioner envia criação, atualização e desativação de usuários e grupos para o serviço. [R10]

IdP / Provisioner	Sua aplicação / SCIM API
POST /Users	cria usuário
PATCH /Users/{id}	altera atributos / active=false
POST /Groups	cria grupo
PATCH /Groups/{id}	altera membership
+----->	

8.3 O lifecycle completo

1. HR cria colaborador -> identidade nasce no diretório corporativo.
2. Usuário entra em grupos e recebe políticas.
3. SCIM/JIT cria a representação na aplicação.
4. Usuário autentica via SSO.
5. Claims/grupos são mapeados para papéis ou atributos internos.
6. Mudança de equipe altera grupos/atributos.
7. Offboarding desativa a identidade e o provisionamento propaga active=false/removals.
8. Sessões/tokens existentes precisam expirar ou ser revogados conforme requisito de risco.

9. Ciclo de sessão, expiração, refresh e logout

9.1 Existem vários relógios

Relógio	Exemplo de política	Quem controla
Sessão do IdP	Reautenticar após X horas ou risco elevado.	IdP.
Sessão da aplicação	Cookie local por 30 min de inatividade.	Sua aplicação.
Access token	5-60 min, conforme provedor/risco.	Authorization Server.
Refresh token	Horas/dias, rotação e políticas.	Authorization Server.
MFA freshness	Exigir MFA novamente para ação sensível.	IdP e/ou aplicação via policy/claims.

9.2 Logout local versus logout global

Logout local encerra apenas a sessão da sua aplicação. Se a sessão do IdP continuar ativa, um novo /login pode autenticar o usuário novamente sem senha. Logout do IdP tenta encerrar a sessão central. Single Logout tenta ainda avisar outras aplicações, mas isso depende do protocolo e do suporte de cada participante.

9.3 OIDC logout

OIDC possui especificações de RP-Initiated Logout e mecanismos front-channel/back-channel. No RP-Initiated Logout, o RP redireciona o usuário ao logout endpoint do OP; post-logout redirect precisa ser validado contra URIs registradas. Back-channel permite que o OP envie uma notificação HTTP direta ao RP. [R6]

9.4 Revogação imediata é difícil com tokens auto-contidos

Se a API valida JWT localmente e não consulta o IdP, um access token já emitido pode continuar válido até expirar. Para reduzir janela de risco, sistemas usam tokens curtos, eventos de logout/back-channel, listas de revogação em casos específicos, introspecção ou políticas de sessão. A escolha é um trade-off entre disponibilidade, latência e revogação imediata.

10. Active Directory, LDAP, Kerberos, AD FS e Microsoft Entra ID

10.1 Active Directory Domain Services (AD DS)

AD DS é a implementação de diretório corporativo tradicional da Microsoft. Armazena usuários, computadores, grupos, políticas e informações relacionadas a credenciais. LDAP é um protocolo central de acesso ao diretório; Kerberos é o principal protocolo de autenticação de domínio em ambientes Windows modernos. [R11]

10.2 Kerberos em uma frase

Kerberos permite que um usuário autenticado no domínio obtenha tickets para acessar serviços sem reenviar sua senha a cada serviço. Por isso, dentro de uma rede Windows/domain-joined, o usuário pode experimentar SSO “integrado” antes mesmo de OIDC/SAML entrarem na conversa.

```
[Usuário/Workstation] -- TGT --> [KDC / Domain Controller]
|                               |
| -- pede ticket SPN -->|
|<-- service ticket -----|
|
|+---- service ticket -----> [Servidor / serviço]
```

10.3 LDAP não é “o protocolo de SSO”

Uma aplicação pode fazer bind LDAP para validar credenciais, mas isso significa que a aplicação participa diretamente da autenticação e normalmente precisa lidar com senha. Em um desenho moderno de SSO web, prefira delegar autenticação a um IdP via OIDC/SAML em vez de transformar cada aplicação em cliente LDAP de senha.

10.4 AD FS

Active Directory Federation Services é um serviço de federação que pode emitir tokens para aplicações e federar identidades do AD. Ele foi muito usado para levar identidades on-premises a aplicações federadas. Em estratégias atuais da Microsoft, Entra ID é normalmente preferido quando possível, embora AD FS ainda exista em ambientes e requisitos específicos. [R11]

10.5 Azure AD agora é Microsoft Entra ID

Microsoft renomeou Azure Active Directory para Microsoft Entra ID. O Active Directory on-premises continua chamado Active Directory. Essa distinção é importante: “AD” pode significar o diretório local; “Azure AD/Entra ID” é o serviço cloud de identidade. [R11]

10.6 Identidade híbrida

Em empresas com AD on-premises e serviços cloud, identidades podem ser sincronizadas/federadas para Entra ID. Assim o usuário mantém uma identidade comum em recursos locais e cloud, com diferentes mecanismos por trás. [R11]

11. Microsoft Entra ID como IdP da sua aplicação

Se a empresa usa Microsoft 365/Entra, um cenário muito comum é registrar sua aplicação no tenant e usar OIDC/OAuth 2.0 para login e APIs, ou SAML para aplicações que trabalham nesse protocolo. O Entra também adiciona políticas como MFA e Conditional Access fora do código da aplicação.

11.1 App registration e enterprise application

Em termos práticos, você encontrará conceitos de registro de aplicação, client ID, redirect URIs, secrets/certificados para clientes confidenciais, scopes, app roles, consentimento e service principals. O nome exato das telas varia, então entenda o modelo em vez de decorar menus.

11.2 Fluxo típico OIDC com Entra

1. Sua aplicação redireciona ao authorization endpoint do tenant.
2. Entra autentica usuário e aplica políticas/MFA.
3. Retorna authorization code para redirect URI registrada.
4. Sua aplicação troca code por tokens usando PKCE e, se for cliente confidencial, sua autenticação de cliente.
5. Valida issuer, audience, assinatura, state/nonce e demais claims.
6. Mapeia usuário para a conta local e cria sessão.
7. API valida access token emitido para a audience correta.

Use biblioteca suportada A própria documentação da Microsoft recomenda usar bibliotecas de autenticação suportadas em vez de construir requests e validações de protocolo manualmente. [R11]

12. AWS: IAM Identity Center, STS, IAM e Cognito

Na AWS, identidade de workforce, credenciais temporárias para recursos e identidade de usuários de aplicações são problemas diferentes. Quatro nomes aparecem muito: IAM Identity Center, AWS STS, IAM federation e Amazon Cognito.

12.1 IAM Identity Center (antigo AWS SSO)

IAM Identity Center centraliza acesso de workforce a contas e aplicações AWS. Pode usar sua própria fonte de identidades, Microsoft AD ou um IdP externo. Com IdP externo, a AWS documenta integração via SAML 2.0 para autenticação e SCIM para provisionamento de usuários/grupos. [R12][R13]

12.2 AWS STS - Security Token Service

AWS STS emite credenciais de segurança temporárias. Em vez de entregar access keys permanentes para humanos, um fluxo federado pode autenticar o usuário e então assumir uma role, recebendo AccessKeyId, SecretAccessKey e SessionToken temporários. [R14]

```

Identidade corporativa
|
| SAML assertion / OIDC identity
v
[AWS federation / IAM Identity Center / STS]
|
| credenciais temporárias
v
[AWS Role / serviços AWS]
```

AssumeRoleWithSAML e AssumeRoleWithWebIdentity

STS possui operações específicas para federação. AssumeRoleWithSAML troca uma assertion SAML por credenciais temporárias de uma IAM role; AssumeRoleWithWebIdentity faz algo semelhante para identidades web/OIDC compatíveis. [R14]

12.3 Amazon Cognito

Cognito User Pools é um serviço de identidade voltado a usuários de aplicações e pode federar com IdPs SAML/OIDC. Cognito Identity Pools troca identidades federadas por credenciais AWS temporárias. Não confunda Cognito com IAM Identity Center: o primeiro é frequentemente usado para identidade de clientes/aplicações; o segundo para workforce/acesso organizacional. [R15]

12.4 Exemplo Entra ID + AWS

A própria AWS mantém um tutorial de laboratório integrando Microsoft Entra ID ao IAM Identity Center: SAML 2.0 cuida do SSO e SCIM sincroniza usuários/grupos. Esse exemplo é excelente para enxergar a separação entre autenticação e provisionamento. [R13]

13. Ciclos completos que você deve saber desenhar

13.1 OIDC Authorization Code + PKCE - aplicação web

1. Usuário acessa /app.
2. App detecta ausência de sessão local.
3. App gera state, nonce e code_verifier; calcula code_challenge.
4. Browser é redirecionado ao IdP /authorize.
5. IdP verifica sessão central. Se necessário, autentica + MFA.
6. IdP redireciona para callback com code e state.
7. App compara state e troca code + verifier por tokens.
8. App valida ID token e cria/vincula usuário local.
9. App cria cookie de sessão HttpOnly/Secure/SameSite conforme arquitetura.
10. Chamadas à API usam sessão BFF ou access token conforme desenho.
11. Quando access token expira, backend/cliente usa refresh conforme política.
12. Logout encerra sessão local e, quando requerido, aciona logout do IdP.

13.2 SAML SP-initiated - aplicação corporativa

1. Usuário acessa recurso protegido no SP.
2. SP gera AuthnRequest e guarda correlação.
3. Browser vai ao IdP.
4. IdP autentica ou reutiliza sessão existente.
5. IdP emite SAML Response/Assertion assinada para o ACS do SP.
6. SP valida assinatura, certificado, issuer, destination, audience, timestamps e correlação.
7. SP extrai NameID/atributos, resolve conta local e cria sessão.

13.3 Entra ID -> IAM Identity Center

1. Admin configura confiança SAML entre Entra ID e IAM Identity Center.
2. Admin configura SCIM para sincronizar usuários e grupos.
3. Usuário entra no portal corporativo com Entra.
4. Entra emite SAML assertion para IAM Identity Center.
5. IAM Identity Center reconhece usuário já provisionado e exibe contas/roles permitidas.
6. CLI/portal obtém credenciais temporárias para a role, evitando chaves permanentes de humano.

14. “Como atuar offline?” - o que isso pode significar

Esse é um dos pontos mais importantes da sua anotação. “Offline” pode significar coisas diferentes: IdP fora do ar; internet do usuário indisponível; sua aplicação isolada do IdP; API sem acesso ao JWKS; ou requisito real de permitir uso local sem rede. Cada caso exige uma política diferente.

14.1 IdP indisponível, mas usuário já tem sessão na aplicação

Sua aplicação pode continuar aceitando uma sessão local ainda válida, se a política de risco permitir. O IdP não precisa ser consultado a cada request. Isso aumenta resiliência, mas significa que um bloqueio central pode demorar até a sessão/token local expirar ou chegar uma notificação de revogação.

14.2 Validar JWT quando o IdP está fora do ar

Se a API já possui as chaves públicas do issuer em cache, ela pode validar assinatura e claims localmente enquanto a chave ainda é considerada confiável. O cache de JWKS deve respeitar rotação e falhas. Não é aceitável desabilitar assinatura ou aceitar token expirado apenas porque o IdP está indisponível.

14.3 Refresh token durante indisponibilidade

Refresh token não resolve a indisponibilidade do Authorization Server: para trocar refresh token por access token você normalmente chama o token endpoint. O scope `offline_access` significa “o usuário não precisa estar presente”, não “o servidor de identidade pode estar desligado”. [R4]

14.4 Usuário completamente offline

Se a aplicação precisa funcionar sem rede, isso se torna um requisito próprio de produto: você pode manter dados locais e uma sessão offline previamente estabelecida, com expiração curta, armazenamento criptografado e conjunto limitado de ações. Isso não é SSO. No retorno da conectividade, a aplicação revalida identidade/autorização e sincroniza ações conforme regras de conflito.

14.5 Estratégias de degradação

Estratégia	Vantagem	Risco/limite
Continuar sessão local válida	Alta disponibilidade.	Revogação central não é instantânea.
Cache de JWKS	API valida tokens sem chamada por request.	Precisa política de rotação/falha; não ajuda token expirado.
Token curto + sessão BFF longa controlada	Reduz exposição do bearer no browser.	BFF precisa disponibilidade e política própria.
Modo offline limitado	Permite operação local temporária.	Complexidade, dados desatualizados e risco de autorização stale.
Fallback de login local separado	Pode salvar operações críticas.	Cria segunda superfície de credenciais; deve ser excepcional e fortemente controlado.

Princípio de segurança Indisponibilidade não deve transformar “falha ao verificar” em “aceita mesmo assim”. Defina explicitamente quais estados podem continuar, por quanto tempo e com quais operações.

15. Arquiteturas de aplicação para SSO

15.1 Aplicação server-side tradicional

```

Browser <--cookie sessão--> Web App / Backend
      |
      +----- OIDC/SAML -----> IdP
      |
      +----- access token -> API externa
  
```

É um modelo simples para segurança de browser: tokens podem ficar apenas no servidor e o browser recebe cookie de sessão. CSRF, cookie flags, session fixation e armazenamento de refresh tokens ainda precisam de atenção.

15.2 SPA falando diretamente com IdP/API

A SPA usa Authorization Code + PKCE e recebe access tokens no contexto do navegador. É uma arquitetura comum, porém aumenta a preocupação com XSS e exposição de bearer tokens. Evite localStorage para segredos de longa duração sem entender o risco; minimize lifetime e siga a biblioteca/provedor.

15.3 BFF - Backend for Frontend

```

Browser -- HttpOnly session cookie --> BFF ---- access token ----> APIs
      |
      +----- OIDC -----> IdP
  
```

No padrão BFF, o backend conduz o OAuth/OIDC e mantém tokens fora do JavaScript da página. O browser opera com cookie de sessão. Isso pode reduzir exposição de bearer tokens, ao custo de estado/backend adicional e necessidade de proteção CSRF/cookies.

15.4 API separada

A API deve validar access token destinado à sua audience, verificar scopes/roles/claims relevantes e aplicar autorização de negócio. A UI estar “logada” não é prova suficiente. Cada request protegido precisa de um principal confiável.

15.5 Multi-tenant

Se vários clientes empresariais conectarem seus próprios IdPs, você precisa modelar tenant -> issuer/configuração -> usuários. Nunca escolha o issuer apenas por claim não validada. Tenha estratégia de descoberta do tenant, allowlist de issuers, validação de audience e isolamento de dados. Account linking deve evitar que duas identidades diferentes se fundam apenas porque possuem o mesmo e-mail.

16. Claims, grupos, RBAC e ABAC

SSO entrega identidade e atributos; autorização transforma isso em decisões. Há dois estilos comuns: RBAC (papéis) e ABAC (atributos). Sistemas reais misturam os dois.

Modelo	Exemplo	Ponto de atenção
RBAC	role=ADMIN pode gerenciar usuários.	Evitar explosão de papéis e depender demais de nomes externos.
ABAC	department=finance e region=BR podem aprovar nota até X.	Regras mais expressivas, mas mais difíceis de auditar.
Grupos do IdP	group=app-finance-readers -> role=FINANCE_READ.	Mapeamento deve ser explícito, versionado e testado.
Permissão interna	user_id 123 possui invoice.approve.	Preserva domínio de autorização dentro da app.

16.1 Não use e-mail como identificador principal imutável

E-mails mudam, podem ser reciclados e possuem semântica de comunicação. Prefira identificadores estáveis do issuer, normalmente a combinação de issuer + subject ou identificador documentado pelo provedor, e armazene o e-mail como atributo mutável.

16.2 Group overage e claims grandes

Provedores podem limitar quantos grupos cabem em um token. Em ambientes grandes, a aplicação pode precisar buscar grupos em API/diretório ou receber apenas app roles/grupos específicos. Não projete autorização assumindo que “todos os grupos sempre virão no JWT”.

17. Segurança: ameaças e controles essenciais

A superfície de identidade é uma das mais sensíveis do sistema. O RFC 9700 consolida as práticas atuais de segurança para OAuth 2.0 e substitui vários padrões antigos. [R2]

17.1 Checklist de validação OIDC/OAuth

- HTTPS/TLS em endpoints de produção.
- Redirect URIs registradas de forma estrita; evite wildcards amplos.
- Authorization Code + PKCE; code challenge S256.
- state imprevisível e associado à sessão/transação.
- nonce quando aplicável em OIDC.
- Validar assinatura com algoritmo permitido; não confiar no alg recebido sem política.
- Validar iss, aud, exp, nbf e claims obrigatórias.
- Segregar ID token e access token por finalidade.
- Não registrar tokens/secrets completos em logs.
- Client secrets apenas em clientes capazes de manter segredo; SPAs/mobile são public clients.
- Rotacionar secrets/certificados e suportar rotação de JWKS.
- Refresh tokens protegidos, com rotação/reuse detection quando suportado.

17.2 Ameaças que você deve reconhecer

Ameaça	Exemplo	Controle
Authorization code interception	Atacante rouba code de callback.	PKCE.
CSRF/login CSRF	Callback é associado à sessão errada.	state + cookies/flow corretos.
Token replay	Bearer token roubado é reutilizado.	TLS, lifetimes curtos, proteção do token; sender-constrained quando aplicável.
Open redirect	Redirect URI permite desviar token/code.	Registro exato/allowlist.
XSS	Script rouba token em SPA.	CSP, higiene XSS, BFF quando adequado, minimizar tokens no browser.
Session fixation	Atacante força uma sessão conhecida.	Rotacionar session ID após login e controles do framework.
Algorithm confusion	Validador aceita algoritmo inesperado.	Allowlist de algoritmos e biblioteca madura.
Key confusion/JWK injection	Token direciona para chave controlada pelo atacante.	Obter chaves apenas de metadata/issuer confiável.

17.3 MFA e Conditional Access

Uma vantagem de delegar autenticação a IdP corporativo é permitir que MFA e políticas de acesso sejam aplicadas centralmente. Sua aplicação pode ainda exigir “step-up authentication” para operações sensíveis, pedindo um nível/frescura de autenticação maior em vez de inventar uma segunda senha local.

18. Como implementar sem reinventar o protocolo

18.1 Princípio: biblioteca madura + configuração segura

Não implemente parser/validador SAML, assinatura XML, JWT verification, PKCE ou discovery manualmente se existe biblioteca consolidada para sua stack. O trabalho da equipe deve estar em integrar o protocolo ao domínio da aplicação: sessão, account linking, autorização, provisioning, observabilidade e experiência de erro.

18.2 Camada de identidade sugerida

```
src/
  auth/
    application/
      LoginService
      LogoutService
      SessionService
      IdentityProvisioningService
    domain/
      ExternalIdentity
      AuthenticatedPrincipal
      RoleMapping
      TenantIdentityConfig
    infrastructure/
      oidc/
        OidcClientAdapter
      saml/
        SamlServiceProviderAdapter
    session/
      SessionRepository
    scim/
      ScimController
```

O objetivo dessa separação é evitar que controller, regra de negócio e detalhes de protocolo fiquem misturados. Seu domínio deveria receber um AuthenticatedPrincipal já validado, não um JWT cru que cada use case interpreta de uma forma.

18.3 Dados que costumam existir no banco

Entidade/campo	Exemplo	Observação
users	id=123, name, email, active	Conta interna.
external_identities	provider=enTRA, issuer, subject, user_id	Liga identidade externa a usuário local.
tenants	acme	Cliente/organização.
identity_providers	tenant_id, issuer, client_id, protocol	Configuração por tenant se multi-IdP.
role_mappings	external_group -> internal_role	Mapeamento de autorização.
sessions	session_id, user_id, expires_at, auth_time	Se sessão server-side.
audit_events	login_success, login_failure, logout, mapping_change	Auditoria de identidade.

19. Testes: como provar que o SSO está correto

19.1 Testes unitários

- Mapeamento de claims externas para principal interno.
- Regras de account linking e criação JIT.
- Mapeamento grupos -> roles/permissions.
- Expiração de sessão local.
- Política de fallback/offline.
- Comportamento quando usuário está disabled.
- Tenant/issuer mismatch.

19.2 Testes de integração de protocolo

- Login feliz OIDC/SAML em ambiente de teste.
- Callback com state inválido.
- ID token expirado.
- audience errada.
- issuer errado.
- assinatura inválida / key rotation.
- nonce incorreto.
- redirect URI não cadastrada.
- usuário sem role necessária.
- logout local e logout no IdP.
- refresh token expirado/revogado.

19.3 Testes SAML específicos

- Assertion expirada ou NotBefore futuro.
- Destination/Recipient incorreto.
- Audience incorreta.
- InResponseTo não corresponde à requisição (quando SP-initiated).
- Assinatura ausente ou certificado desconhecido conforme política.
- Duplicate/replay de assertion.
- NameID/attributes ausentes.

19.4 Testes de resiliência/offline

- IdP indisponível durante novo login -> falha controlada, sem bypass inseguro.
- IdP indisponível com sessão local ainda válida -> comportamento conforme política.
- JWKS endpoint indisponível com chave válida em cache -> validação funciona até limite definido.
- kid novo não presente no cache -> refresh controlado; se não houver chave, falha fechada.
- access token expirado durante outage -> não aceitar apenas por indisponibilidade.
- SCIM indisponível -> fila/retry/idempotência conforme arquitetura.

20. Observabilidade e suporte

SSO falha frequentemente em integrações, relógios, configuração de tenant, claims e certificados. Logs precisam permitir diagnóstico sem expor credenciais.

20.1 O que logar

Evento	Campos úteis	Não logar
Login iniciado	correlation_id, tenant, provider, redirect target lógico	code_verifier, secret
Callback	correlation_id, issuer, subject hash/id interno, resultado	ID/access/refresh token completos
Falha de token	kid, issuer esperado, audience esperada, classe do erro	token bruto
SAML failure	entity ID, response correlation, reason code	assertion inteira em produção
Provisionamento	SCIM operation, external ID, result, retry count	bearer token SCIM

20.2 Métricas úteis

- Taxa de login success/failure por IdP e tenant.
- Latência de /authorize callback -> criação de sessão.
- Falhas de token exchange.
- Falhas por invalid_state, invalid_nonce, invalid_audience, unknown_kid.
- Erros SCIM e backlog de provisioning.
- Quantidade de sessões ativas e logouts forçados.

21. Plano de implementação em fases

Fase 0 - Descoberta

- Qual é o IdP corporativo real? Entra ID, AD FS, Okta, Keycloak, outro?
- Qual protocolo será exigido: OIDC, SAML ou ambos?
- O sistema é SPA, SSR, desktop, mobile, API, ou combinação?
- Existe multi-tenant? Cada cliente conecta seu IdP?
- Quais claims/grupos são fonte de autorização?
- Há necessidade de SCIM ou JIT basta inicialmente?
- Qual é o requisito de offline/indisponibilidade exatamente?

Fase 1 - Proof of Concept

1. Criar um ambiente/tenant de teste no IdP.
2. Registrar uma aplicação com redirect URIs de dev.
3. Implementar login/logout com biblioteca oficial/madura.
4. Exibir principal normalizado em uma página protegida.
5. Criar 2-3 grupos/roles e provar mapeamento.
6. Forçar token inválido/expirado e verificar falha fechada.

Fase 2 - Integração de domínio

- External identity mapping.
- Sessão local e revogação.
- RBAC/ABAC.
- JIT/SCIM.
- Auditoria.
- UX de erro e suporte.

Fase 3 - Hardening

- PKCE/state/nonce completos e revisados.
- Rotação de chaves/secrets/certificados testada.
- Política de outage/offline documentada.
- Threat modeling e revisão de logs.
- Testes E2E com IdP real.
- Runbook de incidente de identidade.

22. Laboratórios práticos para fixar o conteúdo

Você aprende identidade muito mais rápido vendo redirecionamentos e tokens reais. Faça os laboratórios em ambiente de teste, nunca com secrets de produção.

Lab A - Inspeccionar um fluxo OIDC

1. Use um IdP de desenvolvimento ou tenant de teste já disponível na empresa.
2. Registre uma aplicação local com callback `http://localhost` apenas para desenvolvimento, se o provedor permitir.
3. Use a biblioteca de sua stack para Authorization Code + PKCE.
4. Abra DevTools > Network e acompanhe `/authorize`, `callback` e a criação da sessão.
5. Decodifique o ID token apenas para estudo e identifique `iss`, `sub`, `aud`, `exp`, `iat`, `nonce`.
6. Localize o documento `.well-known/openid-configuration` e o `jwks_uri`.
7. Faça logout local e observe que o IdP pode continuar autenticado; depois teste logout do IdP.

Lab B - Entra ID + AWS IAM Identity Center

A AWS publica um tutorial oficial de laboratório para integrar Microsoft Entra ID ao IAM Identity Center com SAML + SCIM. Ele é especialmente útil porque mostra autenticação e provisionamento lado a lado. [R13]

Lab C - Teste de falhas

1. Altere o redirect URI para um valor não registrado e observe a rejeição.
2. Altere o audience esperado no seu validador de teste e confirme que token válido para outro cliente é rejeitado.
3. Simule relógio fora de sincronia e observe `exp/nbf`.
4. Simule JWKS indisponível após cache carregado e documente a política do seu SDK.
5. Expire a sessão local mantendo a sessão do IdP: ao entrar novamente, observe SSO sem nova senha.

23. Perguntas inteligentes para levar ao tech leader

Depois desta leitura, estas perguntas ajudam a transformar “estudar SSO” em requisitos concretos do projeto:

1. Nosso IdP será Microsoft Entra ID? Teremos também Active Directory/AD FS on-premises?
2. Vamos suportar OIDC, SAML ou ambos? Qual é obrigatório para a primeira entrega?
3. “AWS” entra no login da nossa aplicação, no acesso de desenvolvedores à infraestrutura ou nos dois?
4. Quando você mencionou AWS, era STS, IAM Identity Center (antigo AWS SSO), Cognito ou outro serviço?
5. Nosso sistema é single-tenant ou cada cliente poderá federar o próprio IdP?
6. Como será o provisionamento: JIT, SCIM ou usuários continuarão sendo cadastrados internamente?
7. Quais grupos/claims externos viram permissões internas?
8. Qual identificador externo vamos usar para vincular usuário: issuer + subject, object ID, outro?
9. Qual é exatamente o requisito “offline”: IdP fora do ar, usuário sem internet ou operação local sem rede?
10. Quanto tempo uma sessão existente pode continuar se o IdP estiver indisponível?
11. Logout precisa ser apenas local ou Single Logout é requisito?
12. Existe exigência de MFA/Conditional Access e step-up para operações sensíveis?
13. Qual biblioteca/SDK a equipe pretende usar na nossa stack?
14. Quais ambientes/tenants de teste teremos para E2E?
15. Precisamos integrar SSO com AWS Console/CLI para workforce ou apenas com a aplicação?

24. Cheat sheet - uma página mental para reunião

Se alguém disser...	Pense imediatamente em...
SSO	Uma autenticação central reutilizada por várias apps; não é um protocolo.
OAuth 2.0	Autorização e access tokens; não é login sozinho.
OIDC	Autenticação/identidade sobre OAuth; ID token + discovery.
SAML	SSO enterprise XML; IdP -> assertion -> SP/ACS.
IdP	Quem autentica e emite assertion/token confiável.
SP / RP	A aplicação que confia no IdP.
JWT	Formato de claims; precisa validação completa.
JWKS	Conjunto de chaves públicas para verificar tokens.
PKCE	Protege authorization code contra interceptação.
state	Correlação/CSRF no fluxo de autorização.
nonce	Liga ID token à requisição OIDC e ajuda contra replay/mix-up.
AD	Diretório on-premises Microsoft; LDAP/Kerberos.
Entra ID	Antigo Azure AD; IdP/identity cloud Microsoft.
STS	Serviço AWS de credenciais temporárias.
IAM Identity Center	SSO/workforce AWS; pode federar IdP externo.
SCIM	Provisionamento de usuários e grupos.
offline_access	Refresh token sem usuário presente; NÃO garante IdP offline.
SLO	Logout coordenado entre IdP e apps; suporte/fiabilidade variam.

25. Glossário essencial

Termo	Definição
ACS	Assertion Consumer Service: endpoint SAML do SP que recebe a resposta.
AD DS	Active Directory Domain Services.
AD FS	Active Directory Federation Services.
Assertion	Afirmação assinada de identidade/atributos, muito usada em SAML.
Audience	Destinatário pretendido de um token/assertion.
Authorization Code	Código temporário trocado por tokens no token endpoint.
Authorization Server	Servidor OAuth que emite tokens.
Bearer Token	Token cujo possuidor consegue usá-lo; deve ser fortemente protegido.
BFF	Backend for Frontend; backend que gerencia tokens e expõe sessão/cookie ao browser.
Claim	Atributo/declaração dentro de token/assertion.
Client	Aplicação OAuth que solicita autorização.
Client ID	Identificador público de uma aplicação registrada.
Client Secret	Credencial confidencial de cliente; não cabe em SPA/mobile como segredo real.
Conditional Access	Políticas de acesso do Entra baseadas em contexto/risco/dispositivo etc.
Discovery	Metadados OIDC publicados pelo provider.
Federation	Confiança entre domínios de identidade.
ID Token	Token OIDC com informações sobre autenticação/usuário para o cliente.
IdP	Identity Provider.
Issuer	Autoridade que emitiu token/assertion.
JIT	Just-in-Time provisioning durante o login.
JWK/JWKS	Chave JSON / conjunto de chaves JSON.
JWT	JSON Web Token.
Kerberos	Protocolo de autenticação baseado em tickets, comum em AD.
LDAP	Protocolo de acesso a diretórios.
MFA	Multi-Factor Authentication.
Nonce	Valor único associado à autenticação OIDC.
OIDC	OpenID Connect.

Termo	Definição
OAuth 2.0	Framework de autorização.
PKCE	Proof Key for Code Exchange.
Principal	Entidade autenticada que executa uma ação.
Refresh Token	Credencial usada para obter novos tokens.
Relying Party	Nome OIDC para aplicação que confia no OpenID Provider.
Resource Server	API protegida por access token.
SAML	Security Assertion Markup Language.
SCIM	System for Cross-domain Identity Management.
Scope	Conjunto de acesso/consentimento solicitado no OAuth.
Service Provider	Nome SAML para aplicação que consome assertion.
SSO	Single Sign-On.
STS	Security Token Service da AWS; emite credenciais temporárias.
Subject/sub	Identificador do sujeito dentro do issuer.
Tenant	Domínio/organização lógica de identidade.

26. Anti-patterns e frases que devem acender alerta

Frase/decisão	Por que é perigoso
“OAuth é nosso login.”	OAuth não padroniza identidade de usuário; use OIDC quando o objetivo é autenticação.
“O JWT decodificou, então é válido.”	Base64 não é verificação de assinatura, issuer, audience ou tempo.
“Vamos aceitar token expirado se o IdP cair.”	Troca indisponibilidade por falha de segurança.
“Vamos guardar refresh token no localStorage.”	XSS pode transformar o token em credencial reutilizável; avalie BFF/armazenamento apropriado.
“E-mail identifica o usuário para sempre.”	E-mail muda/recicla; use identificador estável do issuer.
“SAML autentica, então já sabemos a permissão.”	Autenticação e autorização são problemas distintos.
“Se desativar no IdP, a sessão local some na hora.”	Depende de logout/revogação/lifetime e integração.
“Client secret da SPA está escondido no bundle.”	Código distribuído ao browser não consegue manter segredo.
“Vamos escrever nosso próprio validador XML/JWT.”	Protocolos têm detalhes e histórico de vulnerabilidades; use bibliotecas maduras.

27. Referências oficiais e leituras recomendadas

Fonte	URL oficial
[R1] OAuth 2.0 - RFC 6749	https://www.rfc-editor.org/info/rfc6749/
[R2] Best Current Practice for OAuth 2.0 Security - RFC 9700 (2025)	https://www.rfc-editor.org/info/rfc9700/
[R3] PKCE - RFC 7636	https://www.rfc-editor.org/info/rfc7636/
[R4] OpenID Connect Core 1.0, errata set 2	https://openid.net/specs/openid-connect-core-1_0.html
[R5] OpenID Connect Discovery 1.0	https://openid.net/specs/openid-connect-discovery-1_0.html
[R6] OpenID Connect RP-Initiated Logout 1.0	https://openid.net/specs/openid-connect-rpinitiated-1_0.html
[R7] OASIS SAML 2.0 Technical Overview	https://docs.oasis-open.org/security/saml/Post2.0/sstc-saml-tech-overview-2.0.html
[R8] JSON Web Token - RFC 7519	https://www.rfc-editor.org/info/rfc7519/
[R9] JWT Best Current Practices - RFC 8725	https://www.rfc-editor.org/info/rfc8725/
[R10] SCIM - RFC 7643 e RFC 7644	https://www.rfc-editor.org/info/rfc7643/ https://www.rfc-editor.org/info/rfc7644/
[R11] Microsoft Learn - Entra ID, SSO, OIDC/OAuth, AD/Kerberos	https://learn.microsoft.com/en-us/entra/identity/enterprise-apps/what-is-single-sign-on https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-auth-code-flow https://learn.microsoft.com/en-us/entra/fundamentals/new-name https://learn.microsoft.com/en-us/windows-server/security/kerberos/kerberos-authentication-overview
[R12] AWS IAM Identity Center - external IdPs	https://docs.aws.amazon.com/singlesignon/latest/userguide/manage-your-identity-source-idp.html
[R13] AWS - Microsoft Entra ID + IAM Identity Center (SAML + SCIM lab)	https://docs.aws.amazon.com/singlesignon/latest/userguide/idp-microsoft-entra.html
[R14] AWS STS temporary credentials / federation	https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp.html https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRoleWithSAML.html https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRoleWithWebIdentity.html
[R15] Amazon Cognito federation	https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools-identity-federation.html

Ordem de leitura externa Se você quiser aprofundar após este PDF, leia primeiro OIDC Core (visão geral + Authorization Code), depois RFC 9700, depois a documentação do provedor que a empresa realmente usar. Não comece tentando ler todo SAML Core ou todos os RFCs de ponta a ponta.

28. Resumo final: o que você precisa conseguir explicar amanhã

1. SSO é a experiência de autenticar uma vez e acessar múltiplas aplicações por meio de confiança em uma autoridade de identidade.
2. OAuth 2.0 trata autorização; OIDC adiciona autenticação/identidade; SAML é outro protocolo de federação/SSO muito usado em enterprise.
3. O fluxo moderno mais importante é Authorization Code + PKCE: authorize -> code -> token -> validação -> sessão local.
4. ID token serve ao cliente para identidade/autenticação; access token serve à API; refresh token renova acesso.
5. JWT precisa de validação completa; não basta decodificar.
6. AD, LDAP e Kerberos pertencem ao mundo de diretório/autenticação corporativa on-premises; Entra ID é o serviço cloud de identidade da Microsoft.
7. AWS IAM Identity Center resolve SSO/workforce; AWS STS emite credenciais temporárias; Cognito resolve outros cenários de identidade de aplicação.
8. SCIM provisiona usuários e grupos; SSO sozinho não resolve lifecycle.
9. “Offline” exige distinguir sessão já existente, validação local com JWKS, refresh sem usuário e operação realmente sem rede.
10. A aplicação continua responsável por sessão local, mapeamento de identidade, autorização, segurança, logs, testes e experiência de erro.

Se você lembrar de uma única frase “Eu não quero que minha aplicação valide a senha corporativa; quero que ela confie em um IdP, valide corretamente o artefato de federação e transforme essa identidade externa em um principal interno com autorização explícita.”